



UNIVERSITAT  
POLITÈCNICA  
DE VALÈNCIA

# Python para Ciencia de Datos

## **Tema 4** **Estructuras de Control**

Máster F.P. En IA & Big Data ANALYTICS  
Curso 2026-2027

**UPV**

# Resultados de aprendizaje

- Describir los **flujos de control** más habituales en programación: secuencia, selección y repetición, y las estructuras de control correspondientes en Python
- Identificar la **indentación** como un elemento sintáctico clave en Python para estructurar el código en bloques dentro de las sentencias de control
- Describir la **sintaxis** de las sentencias de control de flujo de Python: selección condicional (if), gestión de excepciones (try) y bucles (for, while)
- **Utilizar** adecuadamente las **sentencias de control** de flujo en el contexto de programas sencillos en Python, incluyendo selección, bucles y gestión de excepciones
- Resolver problemas sencillos de **recorrido** y **búsqueda** en estructuras de datos mediante bucles

# Índice



- 1. Sentencias de control de flujo**
- 2. Selección condicional**
- 3. Gestión de excepciones**
- 4. Iteración y bucles**
- 5. Recorridos y búsquedas**

# Índice



- 1. Sentencias de control de flujo**
- 2. Selección condicional**
- 3. Gestión de excepciones**
- 4. Iteración y bucles**
- 5. Recorridos y búsquedas**

# Estructuras de control de flujo

## Programa secuencial

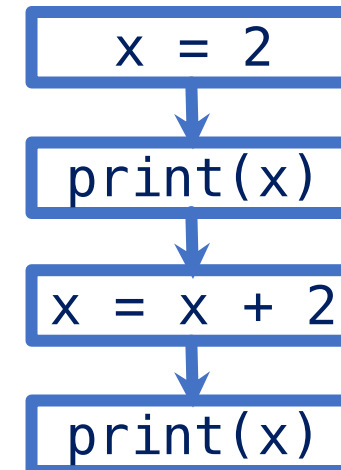
- Hasta ahora, el código que hemos escrito se ajusta a un flujo de control de **programa secuencial**, en el que las instrucciones se ejecutan **una tras otra**
- Por ejemplo:

**Programa:**

```
x = 2  
print(x)  
x = x + 2  
print(x)
```

**Salida:**

2  
4



# Estructuras de control de flujo

## Limitaciones de la instrucción secuencial

- La programación con **instrucciones secuenciales** exclusivamente **limita** de manera significativa la capacidad de resolver problemas
- Por ejemplo, para resolver problemas reales, es necesario que el programa pueda **elegir** entre varias opciones, en función del contexto
- También resulta útil poder **repetir** de manera controlada ciertas instrucciones, en lugar de tener que escribirlas una y otra vez de manera secuencial
- En este tema introduciremos ciertas sentencias que permiten **alterar el flujo** secuencial del programa

# Estructuras de control de flujo

## Estructuras de control en Python

- Las **estructuras de control de flujo** se utilizan para dirigir el **orden** de ejecución de las sentencias en un programa
- Permiten la **bifurcación**, el **bucle** (repetición) y el **control** del flujo en función de diferentes condiciones y/o excepciones
- A continuación, veremos cómo funcionan conceptualmente
- Y luego presentaremos las **sentencias** que nos ofrece Python:
  - **Condicionales:** `if; elif; else`
  - **Bucles:** `while; for; break; continue; pass`
  - **Excepciones:** `try; except; finally; raise`

# Estructuras de control de flujo

## Flujo condicional

- Esta estructura permite ejecutar ciertas instrucciones (o no hacerlo) en función de la evaluación de una **expresión condicional** (booleana)

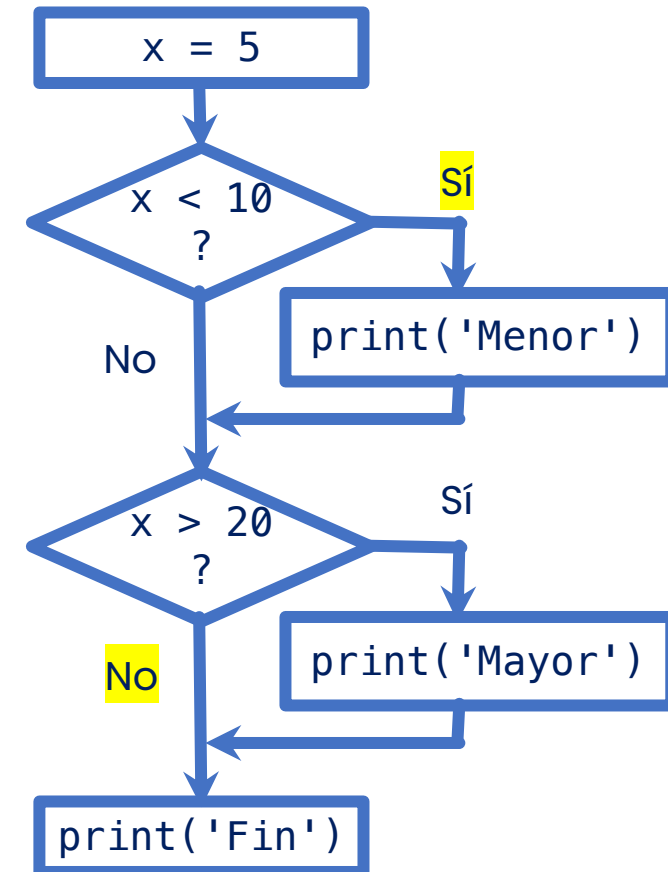
**Programa:**

```
x = 5
if x < 10:
    print('Menor')
if x > 20:
    print('Mayor')
print('Fin')
```

**Salida:**

Menor

Fin





# Estructuras de control de flujo

## Flujo de iteración (bucle)

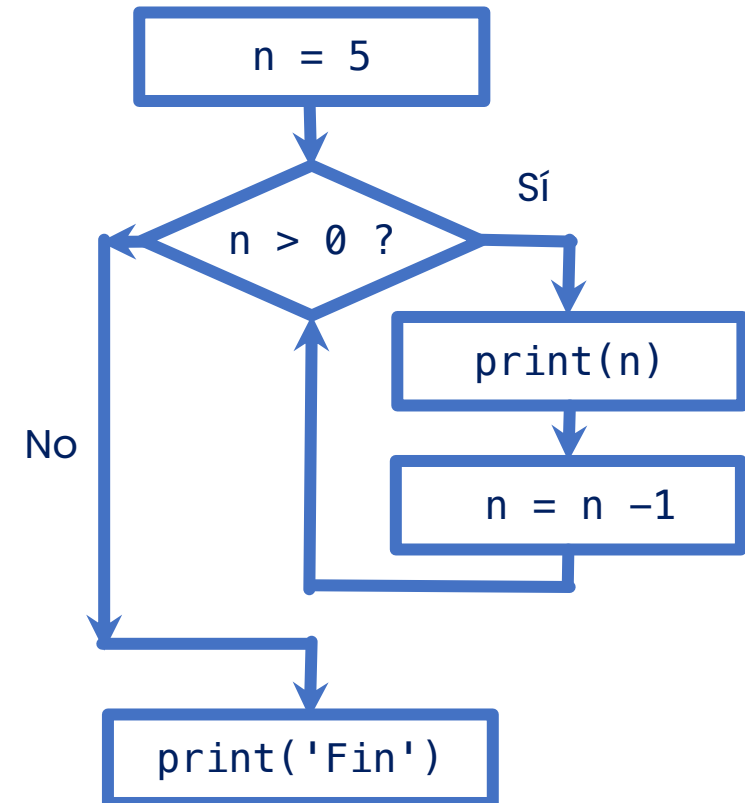
- Esta estructura permite ejecutar ciertas instrucciones de manera **repetida** (bucle) en función de una expresión **condicional** o del recorrido de una **secuencia**

### Programa:

```
n = 5  
while n > 0 :  
    print(n)  
    n = n - 1  
print('Fin')
```

### Salida:

```
5  
4  
3  
2  
1  
Fin
```



# Índice



- 1. Sentencias de control de flujo**
- 2. Selección condicional**
- 3. Gestión de excepciones**
- 4. Iteración y bucles**
- 5. Recorridos y búsquedas**

# Selección condicional

## Selección simple

- Python ofrece una única **sentencia condicional** que puede usarse de varias formas. La forma más sencilla es la **selección simple**:

```
if condición:  
    instrucción1  
    instrucción2  
    ...
```

Las instrucciones del “**bloque del if**” (una o varias) se ejecutan si la condición se evalúa a **True**

```
else:  
    otra_instrucción1  
    otra_instrucción2  
    ...
```

El “**bloque del else**” es **opcional**. Sus instrucciones (una o varias) se ejecutan si la condición se evalúa a **False**

# Selección condicional

## Selección simple

- **MUY IMPORTANTE:** Las instrucciones de cada bloque se deben escribir con una **indentación** respecto al inicio de la línea de las instrucciones `if` o `else`
- **Todas** las instrucciones de un bloque deben tener la **misma indentación** para que Python las interprete correctamente
- La indentación recomendada por Python es de **4 espacios en blanco**
- Los editores de código en Python los suelen insertar automáticamente

```
if condición:  
    instrucción1  
    instrucción2  
    ...  
  
else:  
    otra_instrucción1  
    otra_instrucción2  
    ...
```

# Selección condicional

## Selección simple

- Ejemplos:

```
if x < 10:  
    print('Menor que 10')
```

```
if x < 10:  
    print('Menor que 10')  
else:  
    print('Mayor o igual a 10')
```

- **Recuerda:**

- La condición del `if` es cualquier **expresión booleana**
- Los bloques de `if/else` se componen de una o más instrucciones
- Todo el bloque del `else` es opcional
- Dichos bloques están siempre **indentados**

# Selección condicional

## Selección simple

- **Ejemplo:** Obtener en la variable max el mayor de dos enteros x e y

- 3 alternativas:

```
if x > y:  
    max = x  
else:  
    max = y
```

```
if x >= y:  
    max = x  
else:  
    max = y
```

```
max = y  
if x > y:  
    max = x
```

```
x = int(input('Introduce el 1er número: '))  
y = int(input('Introduce el 2o número: '))  
if x > y:  
    max = x  
else:  
    max = y  
print('El mayor es:', max)
```

### Salida:

```
Introduce el 1er número: -9  
Introduce el 2o número: 5  
El mayor es: 5
```

# Selección condicional

## Selección simple

- **Ejemplo:** Comprobar si un número es par o impar:

```
num = int(input("Introduce el número: "))  
  
if num % 2 > 0:  
    print("El número es impar")  
else:  
    print("El número es par")
```

### Salida:

```
Introduce el número: 3  
El número es impar
```

```
Introduce el número: 0  
El número es par
```

# Selección condicional

## Selección simple

- **Ejemplo:** Comprobar si una palabra ('ola') aparece en una cadena

```
frase = input("Introduce una frase: ")  
  
if "ola" in frase:  
    print("ola está en la frase")  
else:  
    print("ola no está en la frase")
```

### Salida:

```
Introduce una frase: Hola, ¿como estás?  
ola está en la frase
```

```
Introduce una frase: ¿Qué hora es?  
ola no está en la frase
```



# Selección condicional

## Selección anidada

- En el bloque del `if` o el `else` puede aparecer cualquier sentencia, incluida **otra sentencia de selección**
- Ejemplos:

```
if cond1:  
    if cond2:  
        instr1  
    else:  
        instr2
```

| cond1 | cond2         |               |
|-------|---------------|---------------|
|       | True          | False         |
| True  | <i>instr1</i> | <i>instr2</i> |
| False | --            | --            |

```
if cond1:  
    if cond2:  
        instr1  
    else:  
        instr2  
else:  
    instr3
```

| cond1 | cond2         |               |
|-------|---------------|---------------|
|       | True          | False         |
| True  | <i>instr1</i> | <i>instr2</i> |
| False | <i>instr3</i> | <i>instr3</i> |

# Selección condicional

## Selección anidada

- **Ejemplo:** lee las coordenadas de un punto y verifica si ese punto está en el centro, en el eje X, en el eje Y o en un cuadrante

```
x = int(input('Introduce x: '))
y = int(input('Introduce y: '))

if x == 0:
    if y == 0:
        print("En el centro")
    else:
        print("En y-axis")
else:
    if y == 0:
        print("En x-axis")
    else:
        print("En un cuadrante")
```

```
Introduce x: 0
Introduce y: 0
En el centro
```

```
Introduce x: 0
Introduce y: -5
En y-axis
```

```
Introduce x: 9
Introduce y: 0
En x-axis
```

```
Introduce x: 5
Introduce y: -3
En un cuadrante
```

# Selección condicional

## Selección múltiple

- Algunos lenguajes incorporan sentencias de **selección múltiple** que permiten programar comparaciones **sucesivas**, normalmente excluyentes entre sí
- Python consigue este efecto mediante el **término elif** que puede usarse en sentencias `if` para incorporar **múltiples comparaciones** al **mismo nivel**
- La incorporación de múltiples condiciones puede conseguirse mediante el uso de sentencias `if` **anidadas**, como hemos visto
- Sin embargo, el uso de `elif` suele conseguir un código equivalente pero **más compacto** (con menos indentación) y **más legible**

# Selección condicional

## Selección múltiple

- **Anidamiento** de `if` vs uso de `elif`

```
if cond1:
    instr1
else:
    if cond2:
        instr2
    else:
        if cond3:
            instr3
        else:
            instr4
```

vs

```
if cond1:
    instr1
elif cond2:
    instr2
elif cond3:
    instr3
else:
    instr4
```

| cond1 | cond2 | cond3 | Se ejecuta    |
|-------|-------|-------|---------------|
| True  | --    | -     | <i>instr1</i> |
| False | True  | -     | <i>instr2</i> |
| False | False | True  | <i>instr3</i> |
| False | False | False | <i>instr4</i> |

El uso de `elif` reduce las líneas de código y mejora la legibilidad, con lo que resulta **preferible**

# Selección condicional

## Selección múltiple

- **Ejemplo:** dada una calificación, escribe la nota en letras: si la nota es < 5 suspendido, entre 5 y 7 aprobado, entre 7 y 9 notable, entre 9 y 10 sobresaliente y 10 matrícula.

```
grade = float(input("Introduce la nota: "))
if 0 <= grade < 5:
    print("Suspendido")
elif 5 <= grade < 7:
    print("Aprobado")
elif 7 <= grade < 9:
    print("Notable")
elif 9 <= grade < 10:
    print("Sobresaliente")
elif grade == 10:
    print("Matrícula")
else:
    print("Error")
```

Introduce la nota: 7.4  
Notable

Introduce la nota: 0  
Suspendido

Introduce la nota: 10  
Matrícula

Introduce la nota: -1  
Error

Introduce la nota: 11  
Error

# Índice



- 1. Sentencias de control de flujo**
- 2. Selección condicional**
- 3. Gestión de excepciones**
- 4. Iteración y bucles**
- 5. Recorridos y búsquedas**

# Gestión de excepciones

## Errores sintácticos vs excepciones

- Al programar en Python es posible encontrarnos con dos tipos de errores: **errores sintácticos** y **excepciones** (errores en ejecución)
- Un **error sintáctico** se produce cuando el intérprete de Python detecta que alguna sentencia es **sintácticamente incorrecta**
  - El intérprete **revisa** el código del programa **antes** de empezar a ejecutarlo, y se **detiene** en cuanto descubre un error sintáctico
- Una **excepción** es un error que se produce mientras el intérprete se encuentra **ejecutando** una instrucción de un programa que es sintácticamente correcto
  - Por ejemplo, al evaluar una expresión con una división por 0

# Gestión de excepciones

## Errores sintácticos y excepciones

- Una excepción se considera una **situación anómala** durante la ejecución de un programa, y suele **terminar** su ejecución
- Sin embargo, Python permite **detectar** y **tratar** las excepciones que se producen de manera controlada, evitando esa terminación
  - Aprovechando ese tratamiento, también es posible **generar** excepciones personalizadas (útil en determinadas situaciones)
- Para ello, proporciona la estructura de control **try ... except** que veremos a continuación
- Esta estructura puede **alterar el flujo de control** del programa, pero solo si se produce una excepción



# Gestión de excepciones

## Sentencia try - except

```
try:  
    instrucción1  
    instrucción2  
    ...
```

```
except:  
    tratamiento1  
    tratamiento2  
    ...
```

**Cláusula try:** instrucciones en las que **puede** ocurrir una **excepción** que queremos tratar. Si al ejecutarlas no ocurre ninguna, el resto de las cláusulas se descartan

**Cláusula except:** si en el bloque de try se produce una **excepción**, las demás instrucciones se descartan y se pasa a ejecutar las de este bloque (también denominado "**manejador** de la excepción")

# Gestión de excepciones

## Sentencia try - except

- Ejemplo:

```
1 try:
2     n = int(input("Introduce un número:"))
3     print(f"10/{n} = {10/n:8.3f}")
4     print("try finalizado")
5 except:
6     print("Ha habido un error")
```

```
Introduce un número: 34
10/34 =    0.294
try finalizado
```

```
Introduce un número: 0
Ha habido un error
```

```
Introduce un número: hola
Ha habido un error
```

← **1º caso:** ejecución **sin excepciones**. Al finalizar la ejecución de cláusula try, el except se descarta

← **2º caso:** Se produce una **división por 0** en la línea 3. Se descartan las instrucciones restantes y se pasa a ejecutar las instrucciones del except

← **3º caso:** Similar al anterior. Se produce un error de **conversión de tipos** en la línea 2

# Gestión de excepciones

## Tipos de excepciones

- Las excepciones tienen un **tipo asociado**, lo que permite al programa ejecutar diferentes instrucciones (manejadores) dependiendo del tipo de error producido
- Python reconoce muchos **tipos de excepciones**, por ejemplo:
  - División por cero: **ZeroDivisionError**
  - Acceder a un índice inexistente en una secuencia: **IndexError**
  - Abrir (para leer) un archivo que no existe: **FileNotFoundError**
  - Convertir de tipo un valor no posible: **ValueError**
- En la sentencia `try` se pueden incluir varias cláusulas `except`, con distintos tipos asociados

# Gestión de excepciones

## Sentencia try - except completa

```
try:
    instrucción1
    ...
except [Tipo1]:
    tratamiento1
    ...
except [Tipo2]:
    tratamiento2
    ...
else:
    instrucción_else1
    ...
finally:
    instrucción_final1
    ...
```

← **Cláusula try:** instrucciones donde puede producirse una excepción

← **Cláusula(s) except:** si hay varias, se ejecuta aquella cuyo **tipo** corresponde con el de la excepción producida en el try. Solo se ejecuta una de ellas

← **Cláusula else (opcional):** se ejecuta tras el try si **no** se ha producido ninguna excepción

← **Cláusula finally (opcional):** se ejecuta al final de la sentencia, tanto si se ha producido una excepción como si no

# Gestión de excepciones

## Sentencia try - except completa

- Ejemplo:

```
1 try:
2     n = int(input("Introduce un número:"))
3     resultado = 10/n
4 except ZeroDivisionError:
5     print("El número no puede ser 0")
6 except ValueError:
7     print("Debes introducir un número")
8 else:
9     print(f"10/{n} = {resultado:8.3f}")
10 finally:
11     print("Fin")
```

```
Introduce un número: 34
10/34 =      0.294
Fin
```

```
Introduce un número: 0
El número no puede ser 0
Fin
```

```
Introduce un número: hola
Debes introducir un número
Fin
```

# Gestión de excepciones

## Sentencia raise

- Es posible **generar una excepción** mediante la sentencia raise:

```
raise [TipoExcepción]
```

- Esta sentencia se suele usar para generar **excepciones definidas por el programador**, que luego podemos manejar de manera específica, por ejemplo:

```
try:
    ...
    raise Exception("Orden 66")
    ...
except Exception as e:
    print(f"Ejecutar: {e}")
...
```

Ejecutar: Orden 66

**Exception** es la excepción **genérica** en Python. La podemos personalizar con una cadena, a la que podemos acceder en el except mediante as

# Gestión de excepciones

## Ejemplo final

```
try:
    v1 = float(input("Introduce el valor 1: "))
    v2 = float(input("Introduce el valor 2: "))
    op = input("Introduce la operación + - * /: ")
    if op == "+":
        print("Resultado: ", v1 + v2)
    elif op == "-":
        print("Resultado: ", v1 - v2)
    elif op == "*":
        print("Resultado: ", v1 * v2)
    elif op == "/":
        print("Resultado: ", v1 / v2)
    else:
        raise Exception("Operador incorrecto")
except ValueError:
    print("No es float")
except ZeroDivisionError:
    print("División por cero")
except Exception as e:
    print(e)
finally:
    print("Esto siempre se ejecuta")
```

```
Introduce el valor 1: 34
Introduce el valor 2: 23
Introduce la operación + - * /: +
Resultado: 57.0
Esto siempre se ejecuta
```

```
Introduce el valor 1: 3..5
No es float
Esto siempre se ejecuta
```

```
Introduce el valor 1: 23
Introduce el valor 2: 0
Introduce la operación + - * /: /
División por cero
Esto siempre se ejecuta
```

```
Introduce el valor 1: 34
Introduce el valor 2: 75
Introduce la operación + - * /: %
Operador incorrecto
Esto siempre se ejecuta
```

# Índice



- 1. Sentencias de control de flujo**
- 2. Selección condicional**
- 3. Gestión de excepciones**
- 4. Iteración y bucles**
- 5. Recorridos y búsquedas**



# Iteración y bucles

## Repetición de instrucciones vs bucle

- En ocasiones necesitamos que el programa **repita** una instrucción (o varias) un número de veces o mientras se cumpla una condición
- Los **bucles** son estructuras de control que **repiten** (o **iteran**) la **ejecución de instrucciones** las veces que sea necesario, sin tener que escribir esas instrucciones varias veces en el código
- El número de iteraciones puede **decidirse a priori** o **ser un número variable** que dependa de los datos, en función del programa
- Hay dos cuestiones importantes respecto al **nº de iteraciones**:
  - Debe ser **finito** (problema de la **terminación**)
  - Debe ser el **mínimo** posible (**eficiencia**)

# Iteración y bucles

## Bucles en Python

- Python ofrece **dos sentencias de iteración** para implementar bucles:
  - La **sentencia while**: se utiliza cuando queremos **iterar** en función de una **condición** general (es decir, *mientras* se cumpla una condición)
  - La **sentencia for**: se utiliza cuando queremos **iterar** un **número** determinado de veces, o **recorrer** los elementos de una cierta estructura de datos (secuencia)
- A continuación, las analizaremos con detalle

# Iteración y bucles

## Sentencia `while`

- La sentencia **`while`** se utiliza para construir bucles cuya **finalización** depende de la evaluación de una **condición** (expresión booleana)
- Su **sintaxis** es la siguiente:

```
while condición:  
    instrucción1  
    instrucción2  
    ...
```

```
siguiente_instrucción
```

Se evalúa la condición y, si es **cierta**, se ejecutan las instrucciones del **bloque**, y se **vuelve** a evaluar la condición (y así sucesivamente)

Cuando la condición es **falsa**, se pasa a ejecutar la instrucción **posterior** al bucle

**IMPORTANTE:** hay que asegurarse que en algún momento la condición llegue a ser falsa, para evitar un **bucle infinito**

# Iteración y bucles

## Sentencia while

- Ejemplo:

```
1 i = 1
2 while (i < 100):
3     i = i * 2
4     print(i)
5
6 print("Fin")
7
```

```
2
4
8
16
32
64
128
Fin
```

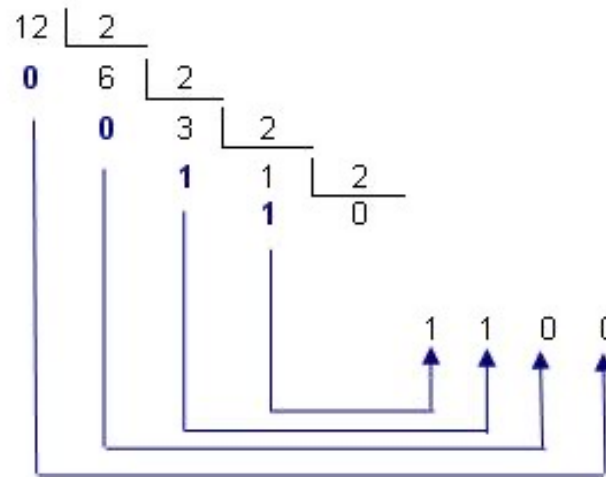
| Iteración | i   | i < 100 | print(i) |
|-----------|-----|---------|----------|
| 1         | 1   | True    | 2        |
| 2         | 2   | True    | 4        |
| 3         | 4   | True    | 8        |
| 4         | 8   | True    | 16       |
| 5         | 16  | True    | 32       |
| 6         | 32  | True    | 64       |
| 7         | 64  | True    | 128      |
| 8         | 128 | False   | --       |

El programa debe asignar un **valor inicial** a la variable `i`, y asegurarse (a priori) que la condición (`i < 100`) **dejará de ser cierta** en algún momento

# Iteración y bucles

## Sentencia `while`

- **Ejemplo:** Convertir un valor entero decimal ( $n$ ) a un número binario (cadena). Estrategia para la solución:
  - Dividir  $n$  entre 2 de manera sucesiva hasta llegar a 0
  - Mientras que  $n // 2$  no sea 0
  - Agregar el resto ( $n \% 2$ ) al (principio del) número binario



# Iteración y bucles

## Sentencia while

- **Ejemplo:** Convertir un valor entero decimal (n) a un número binario

```
1 n = int(input("Decimal a convertir: "))
2 binario = ""
3 print(f"El número decimal {n}", end = " ")
4
5 while (n != 0):
6     binario = str(n % 2) + binario
7     n = n // 2
8
9 print(f"en binario es {binario}")
```

```
Decimal a convertir: 12
El número decimal 12 en binario es 1100
```

| Iteración | n  | n != 0 | binario |
|-----------|----|--------|---------|
| (antes)   | 12 | --     | ""      |
| 1         | 12 | True   | "0"     |
| 2         | 6  | True   | "00"    |
| 3         | 3  | True   | "100"   |
| 4         | 1  | True   | "1100"  |
| 5         | 0  | False  | --      |

# Iteración y bucles

## Sentencia while

- **Ejemplo:** Convertir un valor entero decimal (n) a un número binario

```
1 n = int(input("Decimal a convertir: "))
2 binario = ""
3 print(f"El número decimal {n}", end = " ")
4
5 while (n != 0):
6     binario = str(n % 2) + binario
7     n = n // 2
8
9 print(f"en binario es {binario}")
```

← **Inicialización** (de variables)

← **Guarda** (condición) del bucle  
(condición de parada)

← **Cuerpo** (o bloque) del bucle

```
Decimal a convertir: 12
El número decimal 12 en binario es 1100
```

# Iteración y bucles

## Sentencia break

- La sentencia **break** se utiliza en el bloque de instrucciones de un bucle para **finalizar** la ejecución del bucle (de manera **inmediata**):

```
while condición1:  
    instrucción1  
    ...  
    if condición2:  
        break  
    instrucción2  
    ...  
siguiente_instrucción
```

| condición1 | condición2 | Se ejecuta                                                                                                                     |
|------------|------------|--------------------------------------------------------------------------------------------------------------------------------|
| True       | False      |  <i>instrucción1</i><br><i>instrucción2</i> |
| True       | True       | <i>instrucción1</i><br>>>> <b>SALIR</b> del bucle<<<<br><i>siguiente_instrucción</i>                                           |
| False      |            | <i>siguiente_instrucción</i>                                                                                                   |

La ejecución de **break** produce la **salida inmediata** del bucle



# Iteración y bucles

## Sentencia break

- Ejemplo:

```
1 i = 1
2 while (i < 100):
3     i = i * 2
4     if i == 8:
5         break
6     print(i)
7
8 print("Fin")
```

```
2
4
Fin
```

| Iteración | i | i < 100 | print(i) |
|-----------|---|---------|----------|
| 1         | 1 | True    | 2        |
| 2         | 2 | True    | 4        |
| 3         | 4 | True    | --       |

break

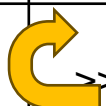
La ejecución de **break** (en este caso, cuando `i == 8`) produce la **salida inmediata** del bucle, y pasa a ejecutarse la instrucción siguiente a `while`

# Iteración y bucles

## Sentencia `continue`

- La sentencia **`continue`** se utiliza en el bloque de instrucciones de un bucle para **finalizar** la iteración (de manera **inmediata**) y volver a evaluar la condición del bucle:

```
while condición1:  
    instrucción1  
    ...  
    if condición2:  
        continue  
    instrucción2  
    ...  
siguiente_instrucción
```

| condición1 | condición2 | Se ejecuta                                                                                                                            |
|------------|------------|---------------------------------------------------------------------------------------------------------------------------------------|
| True       | False      |  <i>instrucción1</i><br><i>instrucción2</i>        |
| True       | True       |  <i>instrucción1</i><br>>>>FIN de la iteración<<< |
| False      |            | <i>siguiente_instrucción</i>                                                                                                          |

La ejecución de **`continue`** produce un **salto inmediato** al **inicio** del bucle (evaluación de la condición)

# Iteración y bucles

## Sentencia `continue`

- Ejemplo:

```
1 i = 1
2 while (i < 100):
3     i = i * 2
4     if i == 8:
5         continue
6     print(i)
7
8 print("Fin")
```

```
2
4
16
32
64
128
Fin
```

| Iteración | i   | i < 100 | print(i) |
|-----------|-----|---------|----------|
| 1         | 1   | True    | 2        |
| 2         | 2   | True    | 4        |
| 3         | 4   | True    | --       |
| 4         | 8   | True    | 16       |
| 5         | 16  | True    | 32       |
| 6         | 32  | True    | 64       |
| 7         | 64  | True    | 128      |
| 8         | 128 | False   | --       |

 **continue**

La ejecución de **`continue`** (cuando `i == 8`) produce el **salto inmediato** a la siguiente iteración: **se omite** el resto de las instrucciones del bloque (`print(i)`) de esa iteración, y se salta al principio del `while`

# Iteración y bucles

## Sentencia `while ... else`

- `while` puede usar de manera **opcional** el término **`else`** para definir un bloque de código que se ejecuta al finalizar el bucle, siempre que este **no** haya sido interrumpido (`break`):

```
while condición:  
    instrucción1  
    instrucción2  
    ...  
else:  
    instrucción_else1  
    ...  
siguiente_instrucción
```

← El bucle `while` funciona de manera normal

Cuando termina el bucle sin interrupciones (la condición es **falsa**) se pasa a ejecutar el bloque del `else` antes de finalizar la sentencia. Si el bucle se interrumpe (por un `break` o excepción), el `else` no se ejecuta

# Iteración y bucles

## Sentencia while ... else

- Ejemplo:

```
1 i = 1
2 while (i < 100):
3     i = i * 2
4     print(i)
5 else:
6     print("Else")
7
8 print("Fin")
```

```
2
4
8
16
32
64
128
Else
Fin
```

```
1 i = 1
2 while (i < 100):
3     i = i * 2
4     if i == 8:
5         break
6     print(i)
7 else:
8     print("Else")
9
10 print("Fin")
```

```
2
4
Fin
```

La instrucción del `else` solo se ejecuta si el bucle finaliza “normalmente”. Si se **interrumpe** por cualquier motivo (`break`), **no se ejecuta**

# Iteración y bucles

## Sentencia for

- La sentencia **for** se utiliza para construir bucles que **recorren**, uno a uno, **los elementos** de una estructura de datos **de tipo secuencia** (como una lista, una cadena, una tupla, etc.)
- Su **sintaxis** es la siguiente:

```
for variable in secuencia:  
    instrucción1  
    instrucción2  
    ...  
siguiente_instrucción
```

En cada **iteración**, la variable toma el valor de un elemento de la secuencia, siguiendo el **mismo orden** en el que se encuentran en dicha secuencia

Cuando se **terminan** los elementos de la secuencia, se pasa a ejecutar la instrucción **posterior** al bucle

# Iteración y bucles

## Sentencia for

- Ejemplo:

```
1 print("Las verduras con mucha vitamina C son:")
2 for nombre in ["Pimiento", "Brócoli", "Espinacas", "Tomates"]:
3     print(nombre)
4 print("Fin")
```

```
Las verduras con mucha vitamina C son:
Pimiento
Brócoli
Espinacas
Tomates
Fin
```

El bucle **recorre** los elementos de la lista (en el mismo orden) y en cada iteración, nombre toma el valor de uno de ellos

# Iteración y bucles

## El tipo de datos range

- El tipo **range** (**rango**) es un tipo de datos **secuencia** cuyo uso está principalmente vinculado a los bucles **for**
- Los **rangos** son **secuencias inmutables de números enteros**, y se definen así:

```
rango = range([inicio,] fin [,incremento])
```

donde:

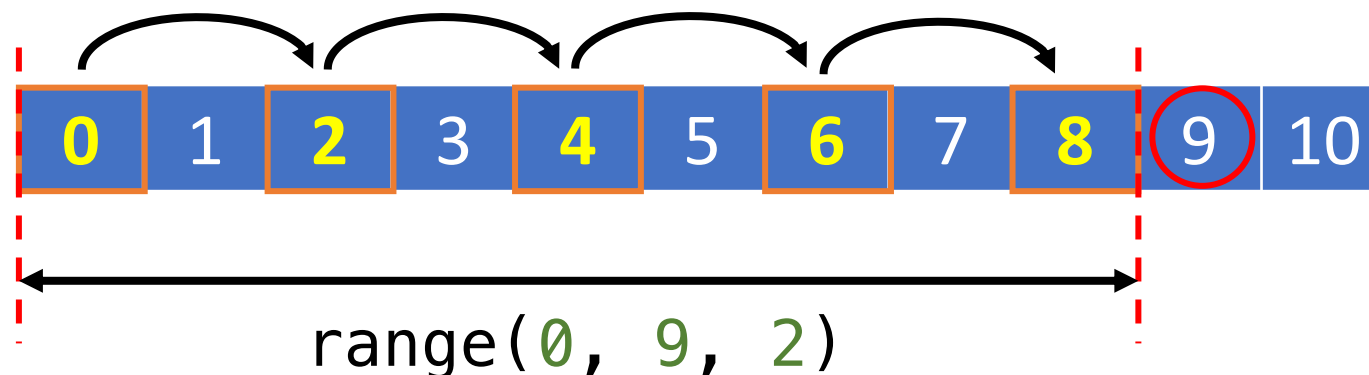
- **inicio**: primer entero de la secuencia (0 por defecto)
- **fin**: número siguiente al último entero de la secuencia
- **incremento**: incremento entre cada número de la secuencia y el siguiente (1 por defecto)



# Iteración y bucles

## El tipo de datos range

- Los rangos **representan** la secuencia de números indicada por sus valores, de la siguiente forma:



La secuencia empieza en valor de inicio y continúa generando números en función del incremento hasta llegar al valor de fin (sin incluirlo)

# Iteración y bucles

## El tipo de datos range

- Ejemplos de rangos:

| Rango                          | Secuencia de números que genera |
|--------------------------------|---------------------------------|
| <code>range(5)</code>          | 0, 1, 2, 3, 4                   |
| <code>range(0, 5, 1)</code>    | 0, 1, 2, 3, 4                   |
| <code>range(1, 6)</code>       | 1, 2, 3, 4, 5                   |
| <code>range(0, 10, 2)</code>   | 0, 2, 4, 6, 8                   |
| <code>range(20, 10, -3)</code> | 20, 17, 14, 11                  |
| <code>range(-1, -7, -2)</code> | -1, -3, -5                      |
| <code>range(0)</code>          | (ninguno)                       |
| <code>range(1, 0)</code>       | (ninguno)                       |

# Iteración y bucles

## El tipo de datos range

- El tipo **range** admite todas las operaciones comunes de las secuencias en Python, excepto la suma (+) y la multiplicación (\*):

| Operador      | Descripción                                  | Ejemplo                  | Resultado   |
|---------------|----------------------------------------------|--------------------------|-------------|
| [índice]      | Operador de indexación (positivo o negativo) | a = range(1,5)<br>a[1]   | 2           |
| [ini:fin:inc] | Operador de corte ( <i>slicing</i> )         | a = range(1,5)<br>a[1:3] | range(2, 4) |
| in (not in)   | Pertenencia (booleano)                       | 3 in range(1,5)          | True        |

| Función        | Descripción              | Ejemplo                  | Resultado |
|----------------|--------------------------|--------------------------|-----------|
| len(rango)     | Longitud (tamaño)        | a = range(1,5)<br>len(a) | 4         |
| max(r), min(r) | Elemento máximo o mínimo | max(a), min(a)           | 4, 1      |

# Iteración y bucles

## El tipo de datos range

- La **diferencia** entre un rango y una lista o tupla es que los rangos no requieren almacenar en memoria los valores que representan
- Cuando es necesario disponer explícitamente de esos valores, el rango puede **convertirse** en una lista (tupla, o conjunto):

`list(range(10))`

`tuple(range(10))`

`set(range(10))`

- Ejemplo:

```
>>> print(range(0, 9, 2))  
range(0, 9, 2)
```

```
>>> print(list(range(0, 9, 2)))  
[0, 2, 4, 6, 8]
```

La lista **contiene** los valores explícitamente

# Iteración y bucles

## Sentencia for con rangos

- Los rangos son la mejor opción para implementar bucles for sobre **secuencias de números**:

```
1 total = 0
2 for i in range(1, 5):
3     print(i)
4     total += i
5
6 print()
7 print(total)
```

```
1
2
3
4
10
```

El bucle **recorre** los **valores** generados por el rango (en el orden en que este los genera) y en cada iteración, `i` toma el valor de cada uno de ellos

Es más **eficiente** que recorrer una lista o tupla, porque no es necesario mantener los valores de la secuencia en memoria

# Iteración y bucles

## Sentencias `break` y `continue` en bucles `for`

- En un bucle `for` también podemos usar `break` y `continue`:

```
1 total = 0
2 for i in range(1, 5):
3     print(i)
4     if i == 3:
5         break
6     total += i
7 print()
8 print(total)
```

1  
2  
3  
  
3

El valor 3 se imprime, pero antes de sumarlo **termina** el bucle:  
 $\text{total} = 1 + 2 = 3$

```
1 total = 0
2 for i in range(1, 5):
3     print(i)
4     if i == 3:
5         continue
6     total += i
7 print()
8 print(total)
```

1  
2  
3  
4  
  
7

El valor 3 se imprime, pero no se suma, y se **continúa** el bucle:  
 $\text{total} = 1 + 2 + 4 = 7$

# Iteración y bucles

## Sentencia for ... else

- Al igual que **while**, **for** puede usar de manera **opcional** el término **else** para definir un bloque de código que se ejecuta al finalizar el bucle, siempre que este **no** haya sido interrumpido:

```
for i in secuencia:  
    instrucción1  
    instrucción2  
    ...  
else:  
    instrucción_else1  
    ...  
siguiente_instrucción
```

← El bucle for funciona de manera normal

Cuando termina el bucle **sin interrupciones** (se ha recorrido toda la secuencia) se ejecuta el bloque del **else** antes de finalizar la sentencia. Si el bucle se **interrumpe** (por un **break** o excepción), **else** no se ejecuta

# Iteración y bucles

## Sentencia for ... else

- Ejemplo:

```
1 for i in range(1, 5):
2     print(i)
3 else:
4     print("Else")
5
6 print("Fin")
```

1  
2  
3  
4  
Else  
Fin

```
1 for i in range(1, 5):
2     print(i)
3     if i == 3:
4         break
5 else:
6     print("Else")
7
8 print("Fin")
```

1  
2  
3  
Fin

La instrucción del `else` solo se ejecuta si el bucle finaliza “normalmente”. Si se **interrumpe** por cualquier motivo (`break`), **no se ejecuta**



# Iteración y bucles

## Equivalencia entre bucles `while` y `for`

- En muchas ocasiones, aunque no siempre, podemos optar por implementar un bucle mediante `while` o `for`:

```
1 total = 0
2 for i in range(1, 5):
3     print(i)
4     total += i
5
6 print(total)
7
```

**es equivalente a**

```
1 total = 0
2 i = 1
3 while i < 5:
4     print(i)
5     total += i
6     i += 1
7
8 print(total)
```

**OJO:** Con `while`, necesitamos **inicializar** la variable `i`, e **incrementarla** en cada iteración

# Índice



- 1. Sentencias de control de flujo**
- 2. Selección condicional**
- 3. Gestión de excepciones**
- 4. Iteración y bucles**
- 5. Recorridos y búsquedas**

# Recorridos y búsquedas

## Recorrido de secuencias: elementos vs. índices

- El principal uso de un bucle for es el recorrido de **secuencias** (listas, tuplas, cadenas, etc.)
- Para recorrer una secuencia siguiendo su **orden de definición** (de izquierda a derecha), podemos hacerlo de dos formas:

```
1 cad = "HOLA"  
2  
3 for letra in cad:  
4     print(letra)  
5
```

H  
O  
L  
A

(1) recorrido **por elemento**: el bucle recorre la propia **secuencia**

```
1 cad = "HOLA"  
2  
3 for i in range(len(cad)):  
4     print(cad[i])  
5
```

H  
O  
L  
A

(2) recorrido **por índice**: usamos un **rango** para recorrer los índices de la secuencia

# Recorridos y búsquedas

## Recorrido de secuencias: elementos vs. índices

- Sin embargo, si queremos recorrer la secuencia en **orden inverso**, solo podemos hacerlo mediante los índices:

```
1 cad = "HOLA"  
2  
3 for i in range(len(a)-1, -1, -1):  
4     print(a[i])  
5
```

A  
L  
O  
H

# Recorridos y búsquedas

## Recorrido de diccionarios

- Para recorrernos un **diccionario** podemos usar un bucle for, y tenemos 3 alternativas: claves, valores o elementos
- **Alternativa 1:** recorrer las **claves** (método `keys()`)

```
1 persona = {  
2     "Nombre" : "Eva Pérez",  
3     "Edad" : 27,  
4     "Altura" : 176  
5 }  
6  
7 for clave in persona.keys():  
8     print(clave, persona[clave])
```

```
Nombre Eva Pérez  
Edad 27  
Altura 176
```

A partir de cada **clave**, podemos acceder también a su valor correspondiente

# Recorridos y búsquedas

## Recorrido de diccionarios

- **Alternativa 2:** recorrer los **valores** (método `values()`)

```
1 persona = {  
2     "Nombre" : "Eva Pérez",  
3     "Edad" : 27,  
4     "Altura" : 176  
5 }  
6  
7 for valor in persona.values():  
8     print(valor)
```

```
Eva Pérez  
27  
176
```

En este caso nos recorreremos los propios **valores**, pero así perdemos referencia de la clave de cada uno

# Recorridos y búsquedas

## Recorrido de diccionarios

- **Alternativa 3:** recorrer los **elementos** (método `items()`)

```
1 persona = {  
2     "Nombre" : "Eva Pérez",  
3     "Edad" : 27,  
4     "Altura" : 176  
5 }  
6  
7 for clave, valor in persona.items():  
8     print(clave, valor)
```

Nombre Eva Pérez  
Edad 27  
Altura 176

El método `items()` devuelve **pares** (clave, valor) que podemos **desempaquetar** directamente en el bucle asignando cada valor a una variable

# Recorridos y búsquedas

## Recorrido de estructuras bidimensionales: anidamiento

- Dentro del bloque de un bucle podemos incluir cualquier sentencia, incluyendo otro bucle (esto se denomina **anidamiento de bucles**)
- Los **bucles anidados** son útiles, entre otros escenarios, para recorrer estructuras **bidimensionales**
- **Ejemplo:** imprimir las tablas de multiplicar del 1 al 10

```
1 for i in range(1, 11):  
2     print(f"Tabla de multiplicar del {i}")  
3     for j in range(1, 11):  
4         print(f"{i:2d} x {j:2d} = {i*j:3d}")  
5
```

Tabla de multiplicar del 1

1 x 1 = 1  
1 x 2 = 2  
1 x 3 = 3  
1 x 4 = 4  
1 x 5 = 5  
1 x 6 = 6  
1 x 7 = 7  
1 x 8 = 8  
1 x 9 = 9  
1 x 10 = 10

Tabla de multiplicar del 2

2 x 1 = 2  
2 x 2 = 4  
2 x 3 = 6  
2 x 4 = 8  
2 x 5 = 10  
2 x 6 = 12  
2 x 7 = 14  
2 x 8 = 16  
2 x 9 = 18  
2 x 10 = 20

Tabla de multiplicar del 3

3 x 1 = 3  
...



# Recorridos y búsquedas

## Recorrido de estructuras bidimensionales: matrices

- Una **matriz** (o lista de listas) es una estructura bidimensional típica, y podemos usar bucles anidados para recorrerlas

```
1 matriz = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]
2
3 for fila in matriz:
4     for item in fila:
5         print(f"{item:2d}", end=" ")
6     print()
```

|    |    |    |
|----|----|----|
| 10 | 20 | 30 |
| 40 | 50 | 60 |
| 70 | 80 | 90 |

Como la matriz es una **lista de filas**, recorreremos en un bucle las filas y, dentro de cada fila, usamos un 2º bucle para recorrer los elementos que contiene

Hemos usado un recorrido por **elementos**: para cada elemento de matriz (fila) y para cada elemento de la fila (item)

# Recorridos y búsquedas

## Recorrido de estructuras bidimensionales: matrices

- Una **matriz** (o lista de listas) es una estructura bidimensional típica, y podemos usar bucles anidados para recorrerlas

```
1 matriz = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]
2
3 for i in range(len(matriz)):
4     for j in range(len(matriz[i])):
5         print(f"({i:1d},{j:1d}) {matriz[i][j]:2d}", end=" ")
6     print()
```

|       |    |       |    |       |    |
|-------|----|-------|----|-------|----|
| (0,0) | 10 | (0,1) | 20 | (0,2) | 30 |
| (1,0) | 40 | (1,1) | 50 | (1,2) | 60 |
| (2,0) | 70 | (2,1) | 80 | (2,2) | 90 |

Esta versión alternativa usa un recorrido por **índices**: entre 0 y la longitud de la matriz (nº de filas), y entre 0 y la longitud de cada fila `i` (`matriz[i]`)

# Recorridos y búsquedas

## Recorrido de estructuras bidimensionales: matrices

- También podemos usar bucles anidados para **construir** una matriz
- Por ejemplo, una “matriz nula” (elementos iguales a cero) de un tamaño predeterminado (n\_filas por n\_columnas)

```
1 matriz = []
2 n_filas = 3
3 n_columnas = 4
4 for i in range(n_filas):
5     fila = []
6     for j in range(n_columnas):
7         fila.append(0)
8     matriz.append(fila)
9 print(matriz)
```

```
1 matriz = []
2 n_filas = 3
3 n_columnas = 4
4 for i in range(n_filas):
5     matriz.append([0] * n_columnas)
6 print(matriz)
```

(versión alternativa con un solo bucle)

```
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

# Recorridos y búsquedas

## Búsqueda en secuencias

- Un algoritmo muy recurrido en programación es el de la **búsqueda** de un elemento concreto en una secuencia recorriendo toda la secuencia hasta encontrarlo (**búsqueda secuencial**)

```
1 lista = [2, 5, 10, 7, 8, 12, 9]
2 x = int(input("Número a buscar:"))
3 pos = -1
4 i = 0
5 while i < len(lista):
6     if x == lista[i]:
7         pos = i
8     i += 1
9 if pos == -1:
10     print(f"{x} no está en lista")
11 else:
12     print(f"La posición de {x} es {pos}")
```

Número a buscar: 8  
La posición de 8 es 4

Número a buscar: 3  
3 no está en la lista

Si lo encontramos, nos guardamos su posición en pos

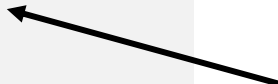
# Recorridos y búsquedas

## Búsqueda en secuencias

- **Ejemplo:** Averiguar el número de palabras de una frase:

```
1 s = input("Escribe una frase: ").strip()
2 if (len(s) > 0):
3     n_palabras = 1
4     for i in range(len(s)):
5         if (s[i] == " "):
6             n_palabras += 1
7 else:
8     n_palabras = 0
9
10 print(f"La frase tiene {n_palabras} palabras")
```

**strip()** elimina los espacios en blanco al inicio y al final de la cadena



```
Escribe una frase: Estoy aprendiendo Python
La frase tiene 3 palabras
```

# Recorridos y búsquedas

## Búsqueda en secuencias

- **Ejemplo:** Mismo ejemplo, usando el método `find()`:

```
1 s = input("Escribe una frase: ").strip()
2 if (len(s) > 0):
3     n_palabras = 1
4     pos = s.find(" ")
5     while (pos != -1):
6         n_palabras += 1
7         pos = s.find(" ", pos + 1)
8 else:
9     n_palabras = 0
10
11 print(f"La frase tiene {n_palabras} palabras")
```

`s.find(c)` encuentra la cadena `c` dentro de `s` y devuelve su posición (o `-1` si no la encuentra)

Si indicamos un número como 2º argumento, busca a partir de esa posición

```
Escribe una frase: Estoy aprendiendo Python
La frase tiene 3 palabras
```

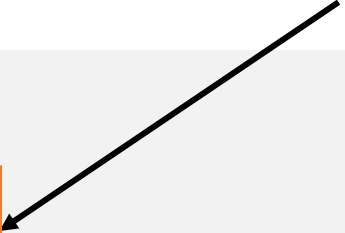
# Recorridos y búsquedas

## Búsqueda en secuencias

- **Ejemplo:** Encontrar el primer número negativo de una lista

Incrementamos `i` mientras `lista[i]` es positivo y aún quedan elementos que recorrer en la lista

```
1 lista = [1, 2, -1, -4, 5, 12]
2 i = 0
3 while i < len(lista) and lista[i] > 0:
4     i += 1
5
6 if i < len(lista):
7     print(f"El número es {lista[i]} y está en la posición {i}")
8 else:
9     print(f"No hay números negativos en la lista")
```



El número es -1 y está en la posición 2.